CineScope Movie Recommendation and Ticket Booking System Using Neo4j

Database Management System Digital Assignment 3

Student	Registration number	Prepared	Demonstration
Durgga DK	25BCE5211	14 October 2026	20 October 2026 lab

Abstract

CineScope extends the earlier relational Movie Booking Management System into a working graph application. Users discover films, record ratings, save watchlists, receive explainable recommendations, and reserve seats for scheduled shows. The app runs locally with Neo4j Desktop and online with Netlify Functions and Neo4j Aura. Both use parameterized Cypher and transactional writes.

The implementation contains nine domain node labels, 18 sample movies, 12 shows, and 144 seats. Recommendations combine shared genres, similar viewers, and sample catalogue ratings. A graph explorer, faculty Query Studio, and committed query playback demonstrate the database. Verification completed 19 live checks on the public site and ten unit checks. Payments and refunds are simulated.

Report scope

This report presents the model, dataset, operations, recommendations, screenshots, verification, and comparison with DA2. Local screenshots and baseline counts are accompanied by cloud deployment and live API test evidence. Live project: https://cinescope-movies-dk28.netlify.app

Requirement from the assessment	Evidence in this submission
Database model and design	Sections 3 and 4; GRAPH_MODEL.md
Implementation and sample data	Sections 2 and 5; runnable source and seed importer
Core and technology specific queries	Sections 6 to 8; Query Studio and Cypher files
Comparative analysis	Section 15
Formal report and project demonstration	This report, implementation screenshots, and DEMO_GUIDE.md

2 Problem and implementation architecture

A movie service contains connected information: a user rates a movie, that movie belongs to genres, another user likes the same movie, and candidate movies can be discovered by following these links. Booking connects a user to a show, screen, theatre, and selected seats. The project implements these connected questions and retains the ticket booking scope established in DA1 and DA2.

The objectives are to build a useful interface, implement the graph model and sample import, demonstrate CRUD and traversals, explain recommendations, and prevent conflicting seat claims. A public Netlify deployment also supports sharing the classroom demonstration.

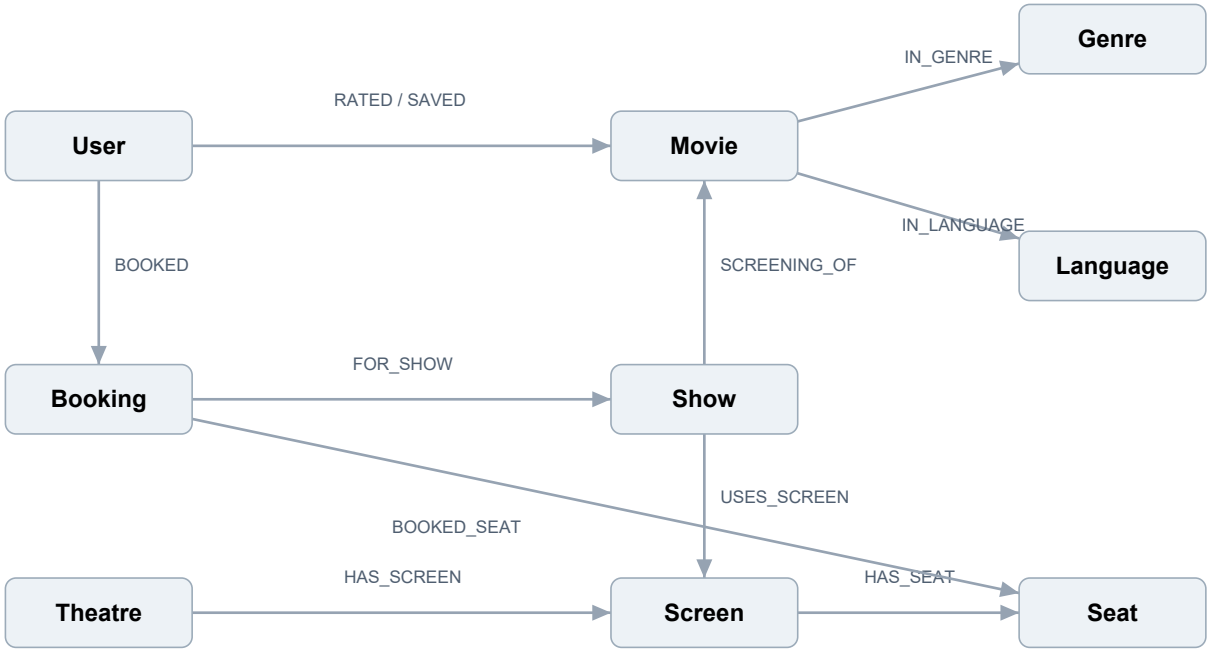
Layer	Implementation
Browser	HTML, CSS, JavaScript modules, SVG artwork and responsive layouts
API	Local Node.js HTTP server; online Netlify Function at /api/*
Database access	Official neo4j-driver 6.2.0; parameterized Cypher and managed transactions
Database	Local Neo4j 2026.09.0; separate Neo4j Aura Free cloud sample graph
Authentication	Salted scrypt hashes, revocable sessions, HttpOnly cookies; Secure online

Mapping the earlier relational project

DA1 or DA2 element	Neo4j implementation
Users and movies	User and Movie nodes
Movie genre and language tables	Genre and Language nodes linked to Movie
Theatre, screen and seat tables	Theatre, Screen and Seat nodes with ownership links
Movie show	Show with SCREENING_OF and USES_SCREEN
Booking and combined payment record	Booking with simulated payment properties
Recommendation table	Derived results and optional RECOMMENDED snapshot edges

FoodBridge informed the modern workspace, database-backed workflow, and query playback approach. CineScope uses its own movie domain model, sample dataset, and recommendation logic.

3 Graph schema and node properties



Label	Main stored properties
User	id, name, email, passwordHash, demo, createdAt
Movie	id, title, year, duration, rating, director, description
Genre and Language	id, name
Theatre	id, name, city, state
Screen	id, name, capacity
Seat	id, number, row, column, type
Show	id, startTime, ticketPrice
Booking	id, status, tickets, amount, paymentMethod, paymentStatus, timestamps

All business nodes also carry the CineScope namespace label and a unique CS-prefixed id. CineAccount supports account email uniqueness. CineSession and the DemoDataset import marker are technical records. They are not additional domain entities and session secrets do not appear in the graph explorer.

4 Relationships constraints and integrity

Source	Relationship	Target	Meaning
User	BOOKED	Booking	User owns the booking
Booking	FOR_SHOW	Show	Booking belongs to a screening
Booking	BOOKED_SEAT	Seat	Selected seats in the screen
Theatre	HAS_SCREEN	Screen	Screen belongs to the theatre
Screen	HAS_SEAT	Seat	Seat belongs to the screen
Show	SCREENING_OF	Movie	Movie shown by the screening
Show	USES_SCREEN	Screen	Screen used by the show
Movie	IN_GENRE	Genre	Genre membership
Movie	IN_LANGUAGE	Language	Film language
User	RATED	Movie	Personal stars and update time
User	SAVED	Movie	Private watchlist and creation time
User	RECOMMENDED	Movie	Optional score and explanation snapshot

Uniqueness constraints

```
CREATE CONSTRAINT cinescope_id IF NOT EXISTS
FOR (n:CineScope) REQUIRE n.id IS UNIQUE;
CREATE CONSTRAINT cinescope_email IF NOT EXISTS
FOR (n:CineAccount) REQUIRE n.email IS UNIQUE;
CREATE CONSTRAINT cinescope_session IF NOT EXISTS
FOR (n:CineSession) REQUIRE n.tokenHash IS UNIQUE;
```

A title index supports catalogue access. The verified schema has three uniqueness constraints; their supporting indexes are created by Neo4j. Required fields, valid ratings, future shows, seat membership, record ownership, and relationship cardinality are validated by the backend. Uniqueness alone does not prevent double booking.

Each booking has one user and one show and between one and six seats. A seat identifier combines its screen and seat number, preserving DA2's composite seat key. Cancelled bookings remain in history and do not block availability.

5 Sample dataset and import

The dataset uses four synthetic accounts and sample cinema schedules. Movie titles identify familiar films; catalogue ratings are example values, not a live ratings feed. Covers are original vector illustrations. No account or database from the reference project is imported.

Domain label	Verified nodes
Booking	3
Genre	10
Language	4
Movie	18
Screen	3
Seat	144
Show	12
Theatre	2
User	4

The domain contains 200 nodes; the import marker adds one technical CineScope node for a total of 201. There are 247 domain relationships before optional recommendation snapshots. Session nodes are tracked separately. Counts are a dated baseline and change after application actions.

Movie ID	Movie	Genres
CS-M101	Interstellar	Sci-Fi, Drama
CS-M102	Inception	Sci-Fi, Thriller
CS-M103	Avatar	Sci-Fi, Adventure
CS-M104	The Dark Knight	Action, Crime
CS-M105	The Martian	Sci-Fi, Drama
CS-M106	Dune	Sci-Fi, Adventure

Arun, Priya, Rahul, and Meera have different sample tastes. Eleven RATED relationships make content and collaborative recommendations demonstrable. Two confirmed bookings occupy three seats across different shows; one cancelled booking demonstrates seat release. Interstellar show CS-SH401 has 46 available seats out of 48.

The explicit importer creates constraints and inserts the sample graph in a managed transaction. A completion marker makes repeated imports idempotent. The 12 sample shows are prepared for 20 to 23 October 2026 in IST. An explicit schedule helper can move these synthetic shows forward for later demonstrations.

6 Explainable graph recommendations

A liked film has a user rating of 4 or 5 stars. The content signal traverses User to liked Movie to Genre to candidate Movie. The collaborative signal follows a shared liked movie to another user and then to that user's liked candidate. Any movie already rated by the current user is excluded.

Genre points equal 12 times the number of matched liked-film and genre paths, capped at 60. Each distinct similar viewer adds 10 points, capped at 30. The sample catalogue rating contributes up to 10 more points. Ranking uses the total, then catalogue rating descending, then title ascending. The score is a rule output, not a probability or trained machine-learning prediction.

Arun recommendation	Genres shared	Viewers	Score
The Martian	Sci-Fi, Drama	1	54
Arrival	Sci-Fi, Drama	1	53.9
Blade Runner 2049	Sci-Fi, Thriller	0	44
Super Deluxe	Drama, Thriller	0	32.2
Avatar	Sci-Fi	0	32

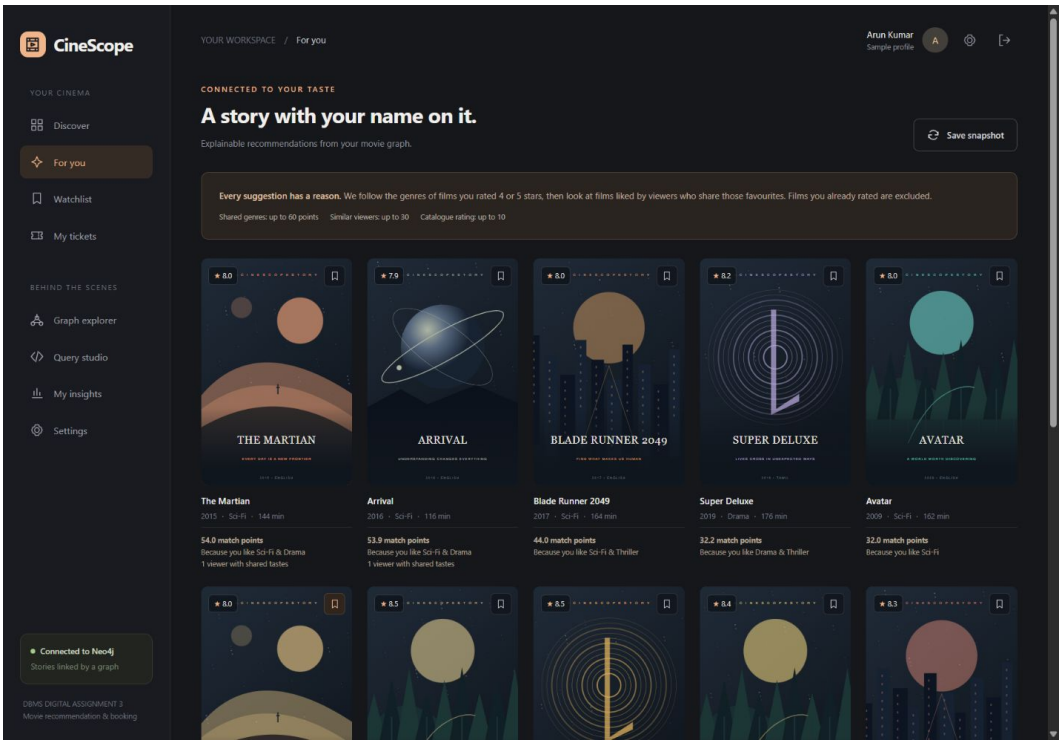


Figure 1. Actual For you results for Arun with visible recommendation reasons.

Save snapshot stores RECOMMENDED edges with score, reason, and generatedAt. Refresh replaces the previous snapshot in one transaction. A new profile without positive rating paths receives a catalogue-rating fallback. Suggestions do not reserve seats.

7 Core operations and booking transactions

The application demonstrates creation through registration and booking, reading through catalogue and show traversal, updating through ratings and cancellation, and deletion through rating or watchlist removal. All values are supplied as parameters. The browser never sends arbitrary Cypher for a business action.

Create or update a personal rating

```
MATCH (u:User:CineScope {id:$userId}),
      (m:Movie:CineScope {id:$movieId})
MERGE (u)-[r:RATED]->(m)
SET r.stars=$stars, r.updatedAt=datetime()
RETURN m.title AS title;
```

Read the show and theatre connection

```
MATCH (m:Movie:CineScope)<-[:SCREENING_OF]-(s:Show)
      -[:USES_SCREEN]->(sc:Screen)<-[:HAS_SCREEN]-(t:Theatre)
RETURN m.title, s.startTime, t.name, sc.name, s.ticketPrice;
```

Delete a rating without deleting the movie

```
MATCH (u:User:CineScope {id:$userId}),
      (m:Movie:CineScope {id:$movieId})
OPTIONAL MATCH (u)-[r:RATED]->(m)
DELETE r RETURN m.title AS title;
```

Seat booking and cancellation

Booking first acquires a write lock on the Show by updating its internal lock property. Inside the same managed transaction it checks the show has not started, all selected seats belong to the screen, and no confirmed booking already holds them. It then creates Booking, BOOKED, FOR_SHOW, and BOOKED_SEAT records atomically. The amount is calculated on the server from the stored show price.

Cancellation checks ownership, takes the same show lock, rechecks status and time, and updates the booking to CANCELLED with a simulated refund. Shared lock ordering makes competing booking and cancellation operations consistent. Live verification sent two requests for the same seat: exactly one succeeded and one received a conflict.

8 Cypher demonstrations and actual outputs

Query Studio example	Purpose
Graph overview	Counts nodes by label
Movies and genres	Traverses the many-to-many genre model
Movie to show to theatre	Follows movie, show, screen, and theatre links
Explain a recommendation	Scores shared genres and viewers
Seat availability	Excludes active seat claims for one show
Booking journey	Traverses the current user's bookings and seats
Confirmed booking totals	Aggregates confirmed sample bookings
Sample users without bookings	Demonstrates NOT EXISTS for synthetic users
Movies related to Interstellar	Finds films with shared genres
Show price aggregation	Computes average, highest, and lowest prices

Availability query

```
MATCH (s:Show:CineScope {id:$showId})
      -[:USES_SCREEN]->(:Screen)-[:HAS_SEAT]->(seat:Seat)
WHERE NOT EXISTS {
  MATCH (b:Booking:CineScope {status:'CONFIRMED'})-[:FOR_SHOW]->(s)
  MATCH (b)-[:BOOKED_SEAT]->(seat)
}
RETURN seat.number AS seat, seat.type AS type ORDER BY seat;
```

With showId CS-SH401 the verified result has 46 available seats. A1 and A2 are occupied; A3 is available because its sample booking is cancelled. The sample show prices are INR 200, 250, 300, and 350, repeated across the 12 shows: average INR 275, highest INR 350, lowest INR 200.

The two confirmed baseline bookings total three tickets and INR 650 in simulated payments. Meera is the baseline sample user without bookings. Each of the ten queries executed successfully during the live verification. Database latency is displayed as a local observation, not a comparative benchmark.

9 Catalogue implementation

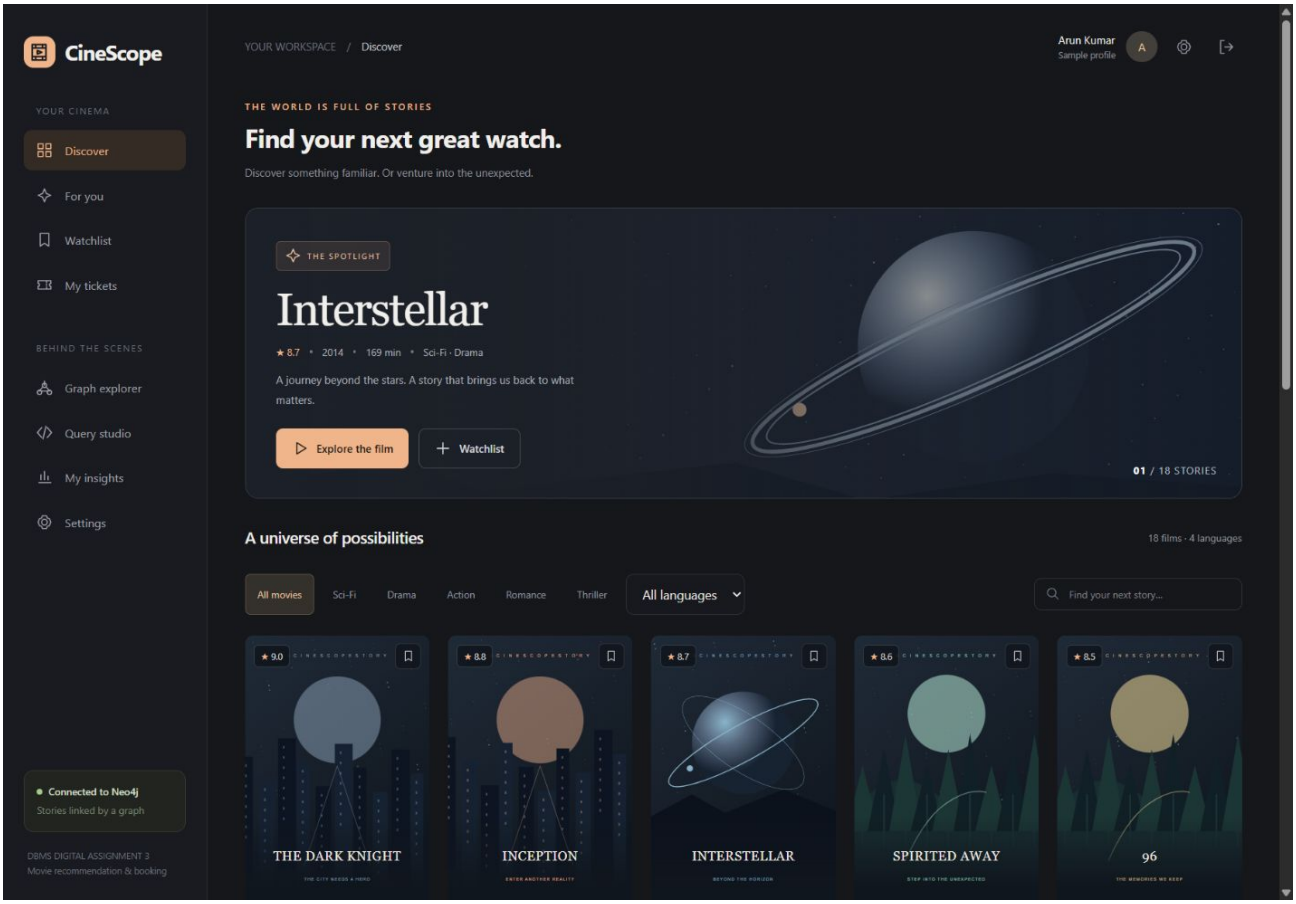


Figure 2. Verified Discover workspace with local illustrative covers and catalogue controls.

Interaction	Implemented behavior
Search	Matches title, director, and language
Filters	Combines genre and language selections
Movie details	Description, sample rating, genres, runtime, and screenings
Personal opinion	1 to 5 stars; removal updates recommendations
Watchlist	Private SAVED relationships persisted in Neo4j
Responsive view	Desktop sidebar and compact mobile navigation

A local system-font stack and vector artwork keep the presentation usable without internet access. Loading, empty, conflict, and error states reflect real API responses. The interface does not show successful saves when a database operation fails.

10 Screening and seat selection

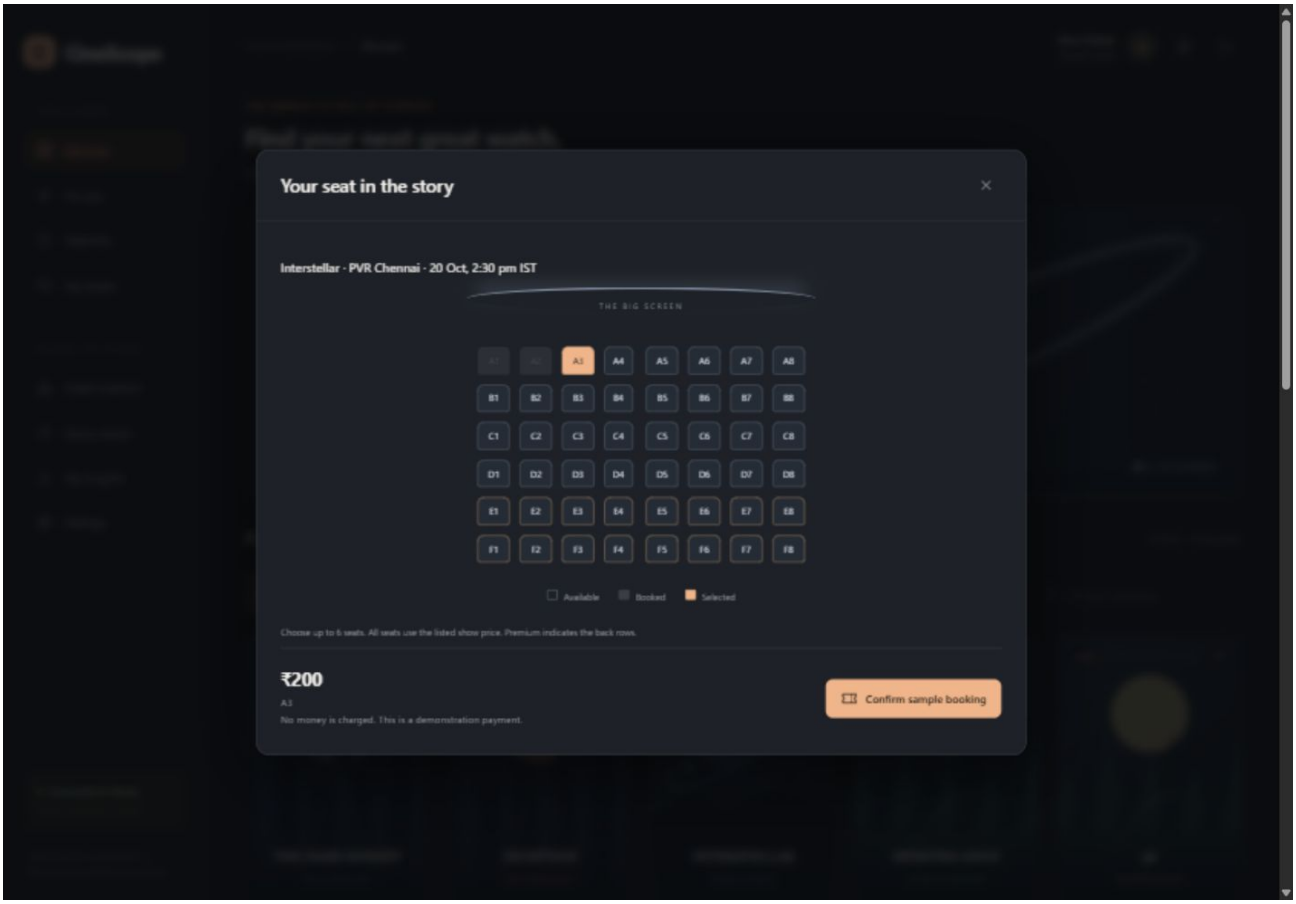


Figure 3. Interstellar seat selection for the October 20 sample show. A3 is selected; the displayed total is INR 200.

The screen has six rows and eight seats per row. A1 and A2 are disabled because they have a confirmed claim for this show. A3 is selectable despite a historical cancelled booking. Premium marks the back rows; every sample seat uses the listed show price.

Selection itself is local interface state. Confirm sample booking sends the selected seat identifiers to the server. The backend rechecks availability while holding the show lock and returns a booking reference only after commit. A stale selection receives a conflict and reloads the map.

The system limits a booking to six distinct seats. Seats from another screen, duplicate seat identifiers, bookings after show start, cancellation after show start, and attempts to cancel another account's booking are rejected.

11 Interactive graph implementation

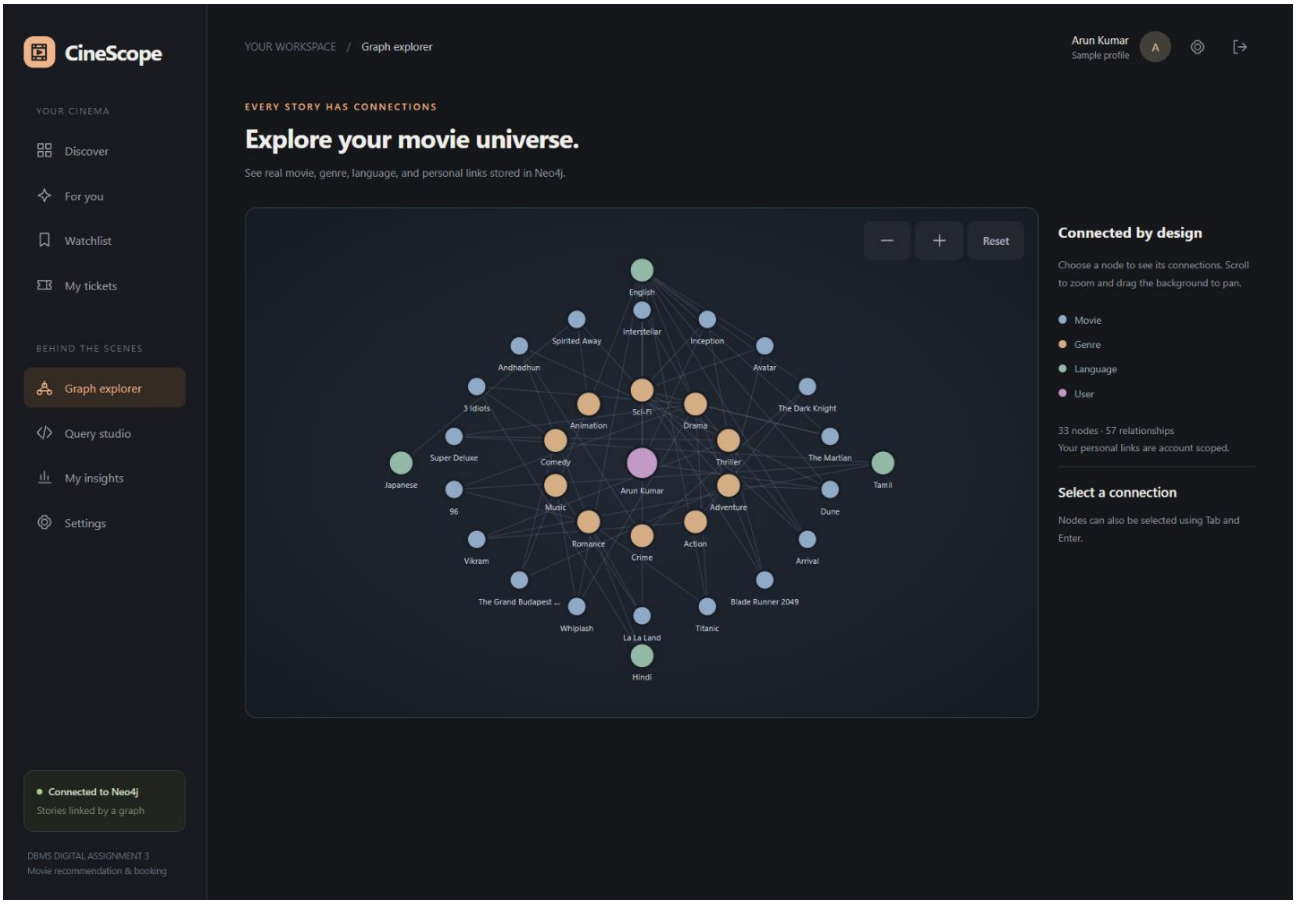


Figure 4. Actual movie, genre, language, and Arun interaction graph from Neo4j.

The visualization queries catalogue relationships plus only the signed-in user's RATED, SAVED, and RECOMMENDED links. It does not expose raw account properties, password hashes, sessions, or another account's private interactions. Baseline Arun graph: 33 displayed nodes and 57 relationships.

Colours distinguish Movie, Genre, Language, and User. A selected node reveals a readable list of its neighbours and relationship types. Controls support zoom, reset, and dragging the background to pan. Nodes can also be selected with Tab and Enter. The graph layout arranges real query results rather than inventing relationships.

The graph explorer focuses on discovery connections. Full booking, screen, theatre, and seat paths remain available in the schema and Query Studio. This avoids placing hundreds of seat nodes in a discovery view.

12 Query execution and committed playback

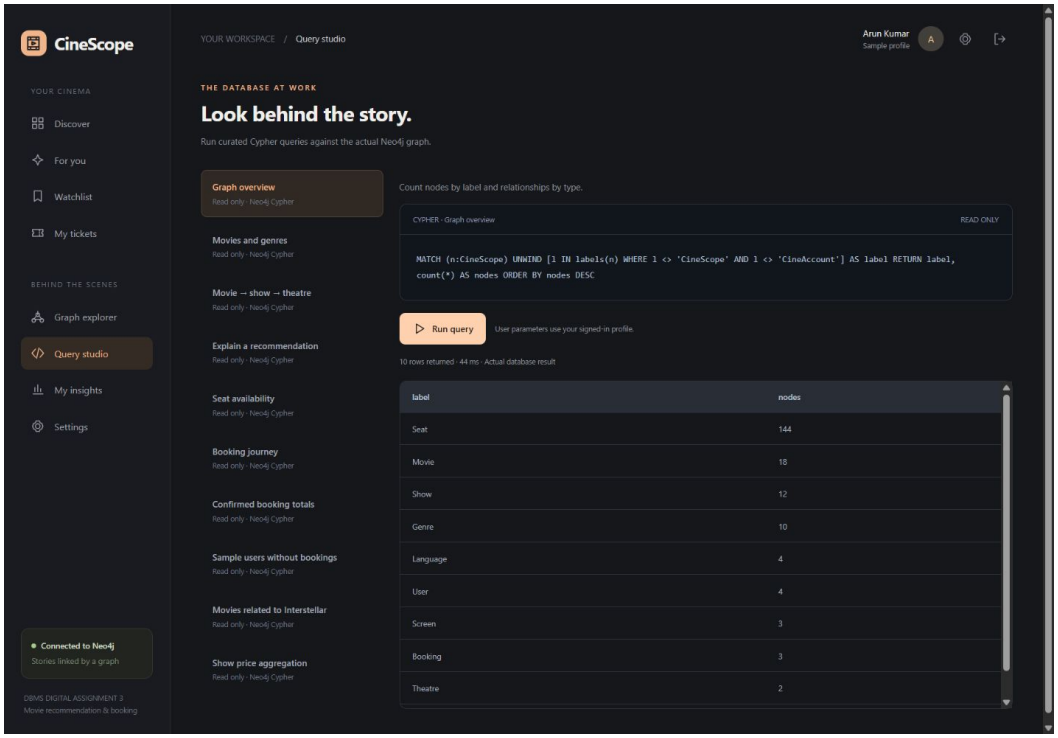


Figure 5. Query Studio executes a curated read-only query and displays the returned rows.

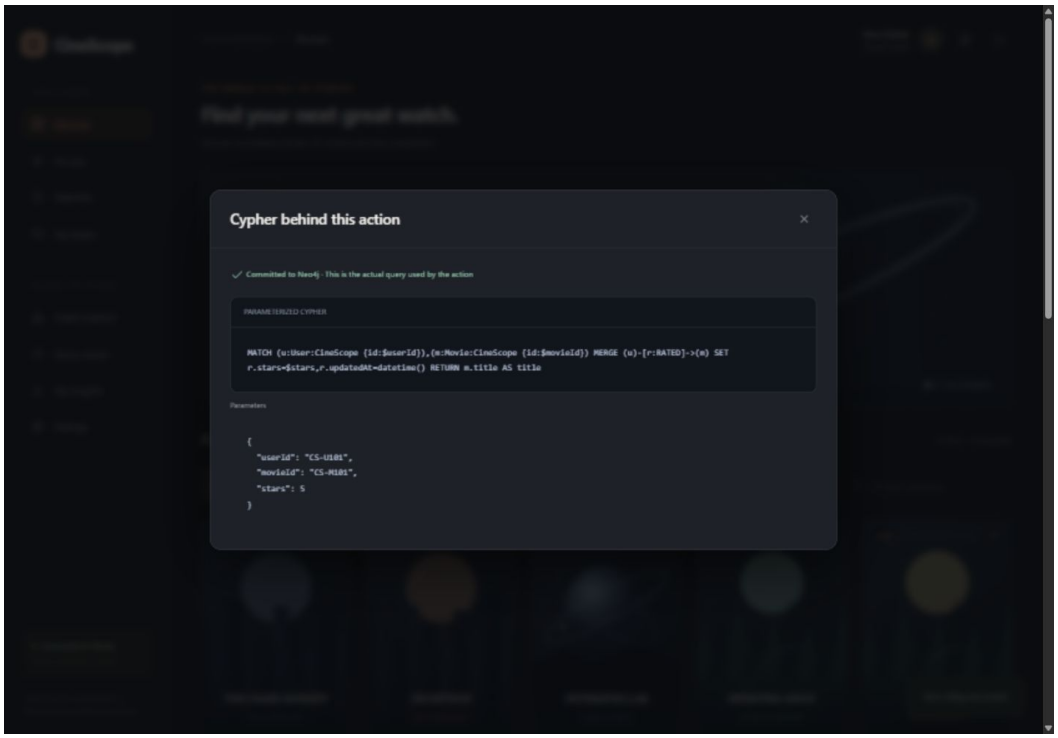


Figure 6. The actual business query and parameters after a successful rating commit.

Query Studio provides a fixed catalogue of ten queries. Playback shows the committed business write; it does not re-execute it. Authentication queries, password hashes, database credentials, and internal locking statements are excluded from the playback.

13 Tickets personal insights and export

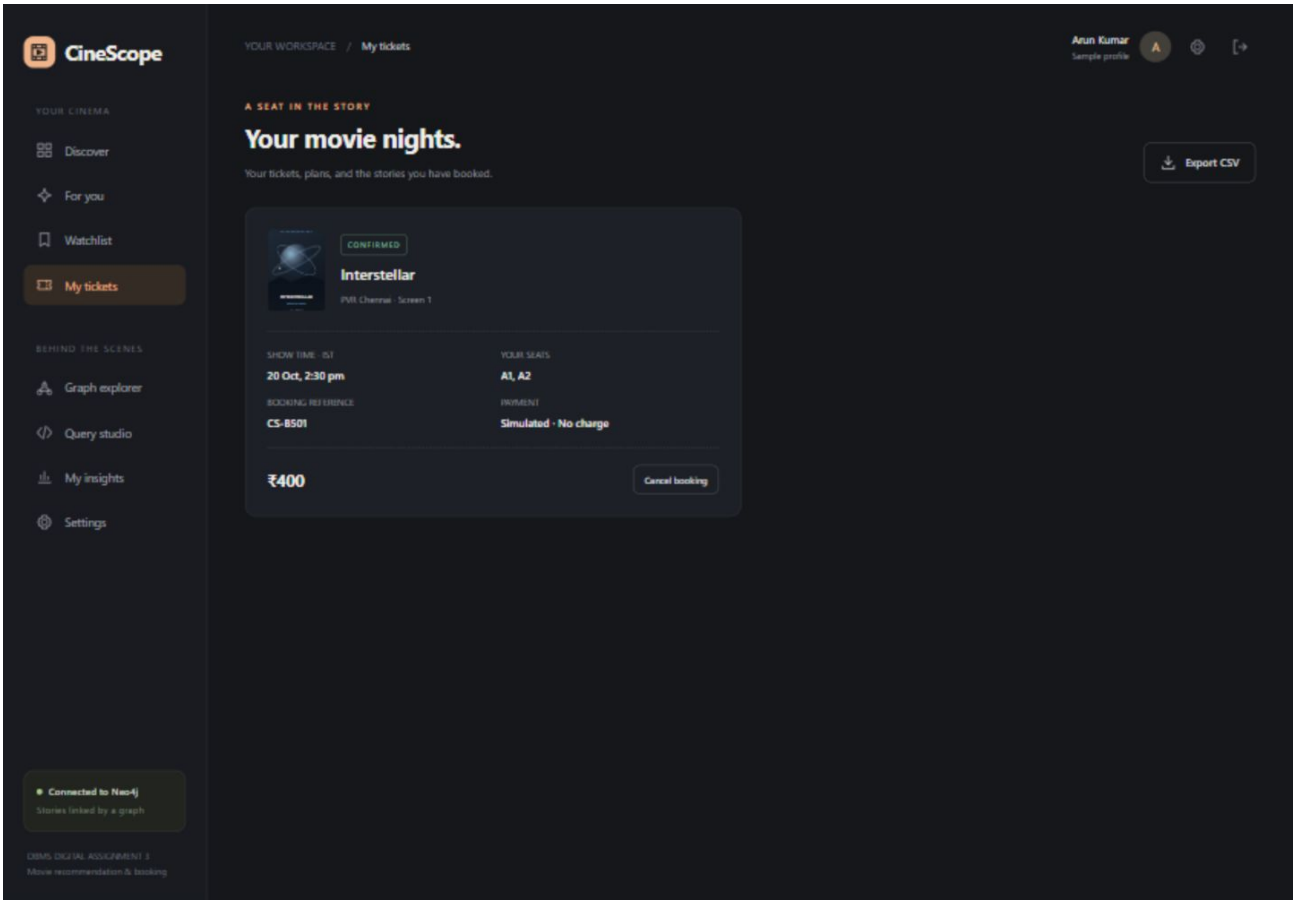


Figure 7. Persisted sample ticket history, including showtime, seats, booking reference, and simulated payment.

My tickets is scoped to the signed-in user through the BOOKED relationship. Cancellation remains available only before a confirmed show starts. The cancelled record remains in history while the seat becomes available to other bookings.

My insights calculates the account's rated film count, saved film count, confirmed booking count, simulated ticket total, and rated genre distribution. A film can belong to multiple genres, so genre totals are not necessarily distinct film totals.

CSV export includes only that account's booking data and no email, password, session, or database credentials. Text cells are escaped and guarded against spreadsheet formulas. These figures describe sample booking activity; they do not represent processed financial transactions.

14 Verification and practical limits

The same 19 live checks passed locally and on the public Netlify API with Neo4j Aura. Disposable fixtures were isolated and removed; sample data was not reset. Cloud results are saved in netlify-production-verification.json. The deployment works independently of the laptop.

Live verification check	Result
Neo4j is connected	Pass
Sample import is idempotent	Pass
Registration and persisted password hashes	Pass
All workspace reads and CSV export succeed	Pass
Anonymous API access is rejected	Pass
Movie catalogue reads from Neo4j	Pass
Rating create update and delete persist	Pass
Watchlists stay private and can be removed	Pass
Recommendations exclude rated films and provide reasons	Pass
Concurrent requests cannot double-book one seat	Pass
Users cannot cancel another account booking	Pass
Duplicate and wrong-screen seats are rejected	Pass
Cancellation releases seats and retains history	Pass
Booking rejects a show that has started	Pass
Graph responses do not expose private account fields	Pass
Every curated Cypher query executes	Pass
Arbitrary Cypher is not accepted as a studio action	Pass
Cross-origin writes are rejected	Pass
Session revocation prevents further access	Pass

Ten unit checks passed, covering validation, password verification, safe CSV, timezone parsing, and hosting safeguards. Browser checks covered desktop/mobile layouts, filtering, seats, tickets, graph controls, and playback. Hosted sign-in and recommendations were also checked in the browser.

This academic demo has no real payments, live cinema feed, email verification, password recovery, or load testing. Cloud credentials stay on the server; public setup and import routes are disabled. Direct database edits can bypass app rules. Tests do not establish production capacity or SQL speed superiority.

15 Database comparison conclusion and demonstration

Aspect	DA2 relational approach	Neo4j approach
Genre membership	Movie and join tables with foreign keys	Explicit IN_GENRE relationships
Recommendation paths	Joins across users, ratings, genres and movies	Cypher matches the connected pattern
Seat transactions	SQL transactions and constraints	Managed transactions plus show write locks
Reporting	Mature SQL aggregation and tabular tooling	Cypher aggregation over graph patterns
Integrity	Strong schema and relational constraints	Uniqueness plus application validation
Performance evidence	No comparative benchmark measured	No universal speed advantage claimed

Neo4j is suitable because the core recommendation question is expressed as a relationship traversal and the result can be explained through the same graph. This is a modelling and query-expression advantage for the selected use case. A relational database can implement the same application and remains a strong choice for tabular reporting and transactions.

Conclusion

The project delivers a working Neo4j-backed recommendation and booking system, sample graph import, core operations, graph-specific queries, explainable results, implementation screenshots, and verified transaction rules. It extends the same mini-project from DA1 and DA2 while using Neo4j as the modern database technology required for DA3.

In person demonstration

On October 20, open the live site or start the local database and app. Sign in as Arun, show recommendations, enable playback, save or rate a movie, explore connections, book and cancel a seat, and run availability queries. Upload the reviewed report to Teams; the app does not submit it automatically.

References

Neo4j constraints: https://neo4j.com/docs/cypher-manual/current/schema/constraints/

Cypher patterns: https://neo4j.com/docs/cypher-manual/current/patterns/

JavaScript managed transactions: https://neo4j.com/docs/javascript-manual/current/transactions/

Concurrent data access and locks: https://neo4j.com/docs/operations-manual/current/database-internals/concurrent-data-access/

Project sources: the supplied DA1 entity model, DA2 SQL assignment, DA3 guideline image, initial movie project specification, and FoodBridge reference brief. Source files, safe Cypher exports, sanitized evidence, and screenshots accompany this report.

16 Public deployment and cloud verification

Live project: https://cinescope-movies-dk28.netlify.app. Select a synthetic sample profile or register a classroom account. The frontend is hosted on Netlify, the API runs in a Netlify Function, and the graph persists in Neo4j Aura Free. The site does not depend on the laptop being switched on.

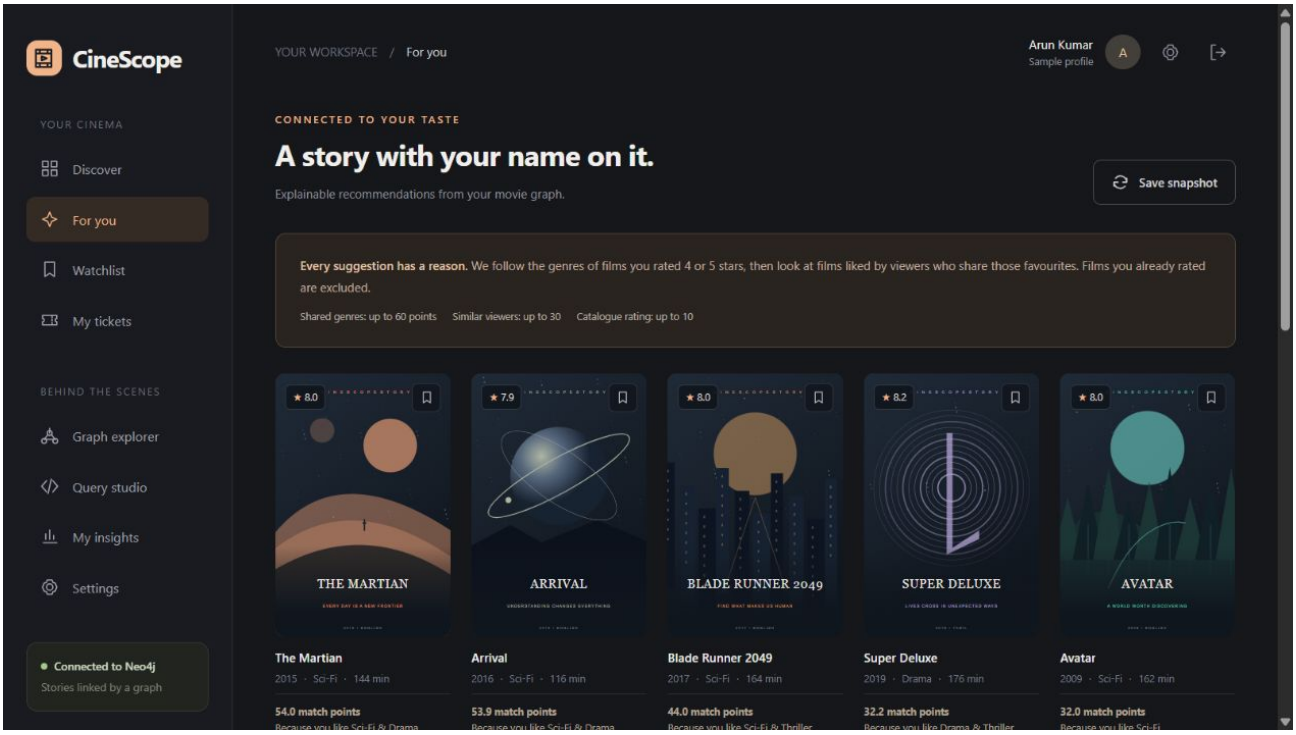


Figure 8. The public Netlify site after Arun signs in. Aura returns The Martian with 54 match points and the shared-taste explanation.

Deployment component	Verified configuration
Public interface	Netlify project cinescope-movies-dk28
Backend	Node.js 24 Function routed to /api/*
Cloud graph	Neo4j Aura Free instance CineScope Movies
End-to-end checks	19 passed against the public URL; test fixtures removed

Cloud and desktop databases contain separate copies of the sample graph. The cloud password is a secret in Netlify server settings. It is absent from frontend files, the function archive, reports, and the submission ZIP. Hosted cookies use Secure, HttpOnly, and SameSite Strict. Public database setup and import routes are disabled.

Netlify Functions: https://docs.netlify.com/build/functions/get-started/

Neo4j Aura connections: https://neo4j.com/docs/aura/getting-started/connect-instance/